

Week5 Session 4 - Facilitation Guide

(Asynchronous Programming & Callbacks)

Index

- I. Recap of the previous session
- II. Synchronous vs. Asynchronous Programming
- III. Callbacks in Node.js
- IV. Callback Hell & How to Avoid It

(2hr ILT)

I. Recap of the previous session

In the previous session, we delved into key foundational concepts of Node.js, a versatile JavaScript runtime environment designed for building scalable and high-performance server-side applications. The focus was on **NPM (Node Package Manager)** and its critical role in the Node.js ecosystem. We started the session by understanding **what NPM is** and how it serves as a package manager that allows developers to access and manage a vast library of reusable modules, tools, and dependencies. We explored the following key concepts:

- **Installing and Using Packages:**
 - We practiced using `npm install` to download and integrate packages into our projects.
 - We understood how `--save` or `--save-dev` flags affect where packages are listed in `package.json`.
- **Understanding `package.json` and `package-lock.json`:**
 - **`package.json`:** We learned how this file acts as the "blueprint" of a Node.js project, specifying the project's dependencies, metadata, and scripts.
 - **`package-lock.json`:** We discussed its role in locking dependency versions to ensure consistency across environments.
- **Managing Dependencies:**
 - We distinguished between **dependencies** (packages required for production) and **devDependencies** (packages needed only for development tasks, such as testing or linting).
 - This distinction allows better organization and optimization of project dependencies.

By the end of the session, we gained hands-on experience with using NPM to efficiently manage Node.js projects and their dependencies.

In today's session, we will dive into **Asynchronous Programming and Callbacks**, a core concept in Node.js that enables it to handle multiple tasks concurrently without blocking the execution thread. By the end of this session, you will have a solid understanding of how to work with asynchronous patterns, including **callbacks**, and how they contribute to the non-blocking architecture of Node.js.

II. Synchronous vs. Asynchronous Programming

Node.js operates on a single-threaded event loop architecture, which makes understanding synchronous and asynchronous programming crucial. These two programming paradigms govern how tasks are executed and how efficiently a Node.js application performs, especially when handling multiple tasks or large workloads.

Synchronous Programming

In synchronous programming, tasks are executed one at a time, sequentially, meaning the current task must complete before the next one begins. If a task takes a long time to finish, the entire program is blocked until that task is completed. This is often referred to as blocking code.

Example of Synchronous Code

```
const fs = require('fs');
// Reading a file synchronously
const data = fs.readFileSync('file.txt', 'utf8');
console.log(data);
console.log('This will print after the file is read.');
```

Explanation:

1. fs.readFileSync reads the file and holds the program execution until the operation is complete.
2. During the file reading, nothing else in the program can run.
3. Only after the file is read and logged to the console does the program proceed to the next console.log() statement.

This approach is simple and easy to understand but becomes inefficient for tasks involving I/O operations (like file reading, database queries, or API calls) because these tasks are time-consuming and can block the entire application.

Asynchronous Programming

In asynchronous programming, tasks can start but don't block the program while waiting to finish. Instead, the program moves on to execute the next task. When the asynchronous task completes, it notifies the program via a callback, promise, or async/await to handle the result.

Asynchronous programming is also called non-blocking code, and it's the core of Node.js' efficiency in handling tasks concurrently.

Example of Asynchronous Code

```
const fs = require('fs');
// Reading a file asynchronously
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
console.log('This will print before the file is read.');
```

Explanation:

1. fs.readFile initiates the file-reading operation and immediately moves on to the next task (the second console.log statement).
2. While the program continues executing other code, the file reading runs in the **background**.
3. Once the file-reading operation is complete, the provided callback function is executed to handle the result.

Asynchronous programming allows Node.js to handle multiple tasks at the same time, making it ideal for **I/O-intensive** applications, like web servers or APIs.

Detailed Differences

Aspect	Synchronous Programming	Asynchronous Programming
Execution Flow	Tasks execute sequentially; one must finish before the next starts.	Tasks execute independently, allowing other tasks to continue while waiting for the current one to complete.
Blocking/Non-Blocking	Blocks the entire program while a task is being executed.	Non-blocking, so the program remains responsive even during long-running tasks.
Performance	Inefficient for I/O-heavy or concurrent operations.	Highly efficient for handling multiple simultaneous operations.
Error Handling	Errors are thrown directly and must be caught explicitly.	Errors can be passed to callbacks, promises, or try-catch blocks in async/await.
Ease of Understanding	Easier to understand due to sequential execution.	Requires understanding of callbacks, promises, and async/await for better readability.
Use Case	Suitable for small, quick tasks where performance isn't a concern.	Best for I/O-heavy tasks like file reading, API calls, or database operations.

Why Does Node.js Favor Asynchronous Programming?

Node.js is designed to be **non-blocking** because it runs on a **single thread**. If one task blocks the thread (e.g., reading a file synchronously), the entire application halts until that task is completed. This is problematic for applications like web servers that handle multiple requests simultaneously.

Example Scenario:

- A synchronous web server handling file uploads would make other users wait until the upload completes.
- An asynchronous web server can handle the file upload in the background while simultaneously serving other requests.

This non-blocking approach is critical for Node.js to handle thousands of requests concurrently, making it highly scalable.

Real-Life Scenario

Synchronous Programming:

Imagine you are in a restaurant with a single chef:

1. The chef works on one dish at a time.
2. If a dish takes 30 minutes to cook, the chef doesn't start the next dish until the first is done.
3. All customers must wait in line, even for simple orders.

Asynchronous Programming:

In contrast, consider a restaurant with the same chef, but now he multitasks:

1. The chef starts cooking a dish but moves on to prepare other orders while waiting for the first dish to finish (e.g., baking in the oven).
2. Orders are managed more efficiently, and customers are served faster.

How Node.js Manages Asynchronous Tasks?

Node.js uses an **event loop** and **callback queue** to handle asynchronous tasks efficiently. Here's how it works:

- When an asynchronous task (e.g., file reading) is initiated, Node.js sends it to the **event loop** and moves to the next line of code.
- The event loop continues running other code while the asynchronous task is executed in the background.
- Once the task is finished, the event loop picks it up and executes the callback or promise.

Tools to Handle Asynchronous Code

Node.js offers various ways to manage asynchronous tasks:

- **Callbacks:** Functions passed as arguments to other functions, executed when a task completes.
- **Promises:** A cleaner way to handle asynchronous tasks.
- **Async/Await:** A modern syntax that makes asynchronous code look synchronous.

Synchronous programming is simple to use but unsuitable for Node.js' single-threaded nature due to its blocking behavior. Asynchronous programming, on the other hand, is the backbone of Node.js' scalability and efficiency. Understanding and mastering asynchronous patterns (like callbacks, promises, and async/await) is essential for building high-performance, non-blocking applications.

III. Callbacks in Node.js

In Node.js, a callback is a function that is passed as an argument to another function. This callback function is executed once the main function has completed its task. Callbacks are a core concept in Node.js because of its asynchronous, non-blocking nature.

The idea is simple: when Node.js performs an asynchronous operation, like reading a file or making a network request, it doesn't wait for the task to complete. Instead, it continues executing other code, and when the asynchronous operation is complete, it "calls back" the provided callback function to handle the result.

Key Characteristics of Callbacks

- **Asynchronous Execution:** Callbacks allow Node.js to handle multiple tasks simultaneously without blocking the event loop.
- **Error Handling:** Callbacks often include an error-first pattern, where the first argument represents an error (if any) and the second argument contains the result.
- **Common Use Cases:** File I/O, database queries, API requests, and timers often use callbacks.

Working Example: Reading a File Asynchronously

Let's demonstrate how callbacks work with Node.js's fs module for reading files.

```
const fs = require("fs");
// Function to read a file using a callback
function readFileWithCallback(filePath, callback) {
  fs.readFile(filePath, "utf8", (err, data) => {
    if (err) {
      callback(err, null); // Pass the error to the callback
    } else {
      callback(null, data); // Pass the file data to the callback
    }
  });
}
// Call the function and provide a callback
readFileWithCallback("example.txt", (error, result) => {
```

```

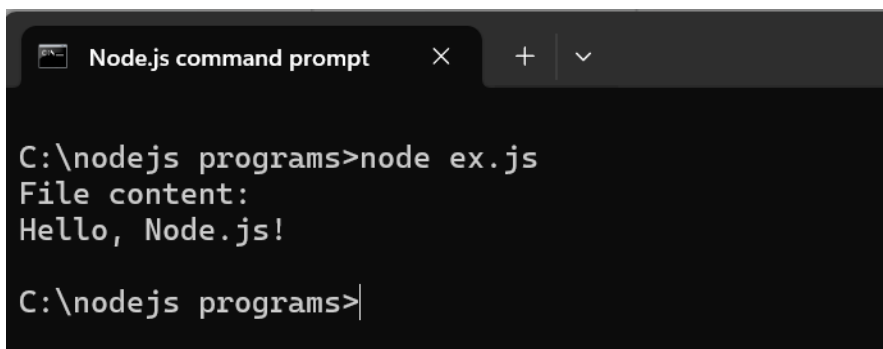
    if (error) {
      console.error("Error reading file:", error);
    } else {
      console.log("File content:");
      console.log(result);
    }
  });
}

```

Explanation:

- `fs.readFile()` : This function is asynchronous and accepts a callback.
- Error-First Callback Pattern:
 - If an error occurs (e.g., file not found), `err` will be non-null, and `data` will be null.
 - If the operation succeeds, `err` will be null, and `data` will contain the file content.
- Callback Handling: The callback function provided handles both the error case and the success case.

Output:



```

Node.js command prompt
C:\nodejs programs>node ex.js
File content:
Hello, Node.js!

C:\nodejs programs>

```

Error-First Callback Pattern

Node.js callbacks follow the "error-first" convention. The first argument to the callback is always an error object (if any), and the subsequent arguments represent the result of the operation.

Example:

```

fs.readFile("nonexistent.txt", "utf8", (err, data) => {
  if (err) {
    console.error("Error occurred:", err.message); // Handle the error
  } else {
    console.log("File content:", data); // Handle the success
  }
});

```

Real-Life Use Case Example: Fetching User Data

Simulate fetching user data with a callback function.

Code Example:

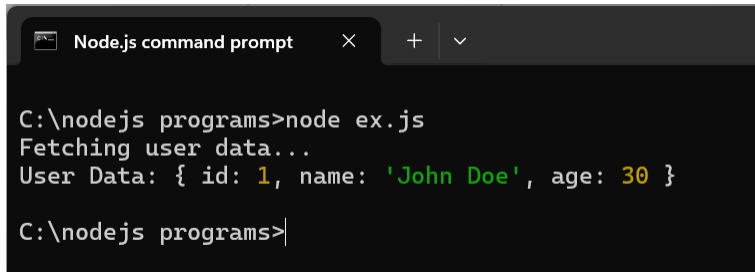
```

function fetchUserData(userId, callback) {
  console.log("Fetching user data...");
  setTimeout(() => {
    if (userId <= 0) {
      callback(new Error("Invalid User ID"), null); // Pass error to callback
    } else {
      const user = { id: userId, name: "John Doe", age: 30 };
      callback(null, user); // Pass user data to callback
    }
  }, 2000);
}

```

```
// Calling the function
fetchUserData(1, (error, user) => {
  if (error) {
    console.error("Error:", error.message);
  } else {
    console.log("User Data:", user);
  }
});
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the following text: "C:\nodejs programs>node ex.js", "Fetching user data...", "User Data: { id: 1, name: 'John Doe', age: 30 }", and "C:\nodejs programs>".

```
C:\nodejs programs>node ex.js
Fetching user data...
User Data: { id: 1, name: 'John Doe', age: 30 }
C:\nodejs programs>
```

Advantages of Callbacks

- Efficient handling of I/O and asynchronous operations.
- Simplifies managing multiple asynchronous tasks.
- Allows Node.js to maintain its non-blocking architecture.

Challenges with Callbacks

- **Callback Hell:** Nesting too many callbacks can make the code unreadable and hard to maintain.
 - Solution: Use Promises or Async/Await.

Callbacks are a fundamental concept in Node.js that allow you to handle asynchronous tasks efficiently. While they are powerful, modern practices like Promises and Async/Await are preferred for better readability and maintainability of code.

IV. Callback Hell & How to Avoid It

Callback hell is a situation in programming, especially in Node.js, where callbacks are nested within other callbacks, forming a pyramid-like structure. This makes the code difficult to read, debug, and maintain.

It occurs when multiple asynchronous operations are dependent on each other and are executed sequentially using callbacks.

Why Does Callback Hell Happen?

- **Deep Nesting:** Asynchronous functions are nested one inside another, leading to an increasing indentation level.
- **Tightly Coupled Code:** Each callback depends on the result of the previous one, making the code fragile and harder to refactor.
- **Poor Readability:** The more nested the callbacks, the harder it is to read and understand the flow of execution.
- **Error Handling Complexity:** Managing errors across multiple nested callbacks becomes cumbersome and inconsistent.

Example of Callback Hell

Here's an example of callback hell where multiple asynchronous file operations are performed:

```
const fs = require('fs');

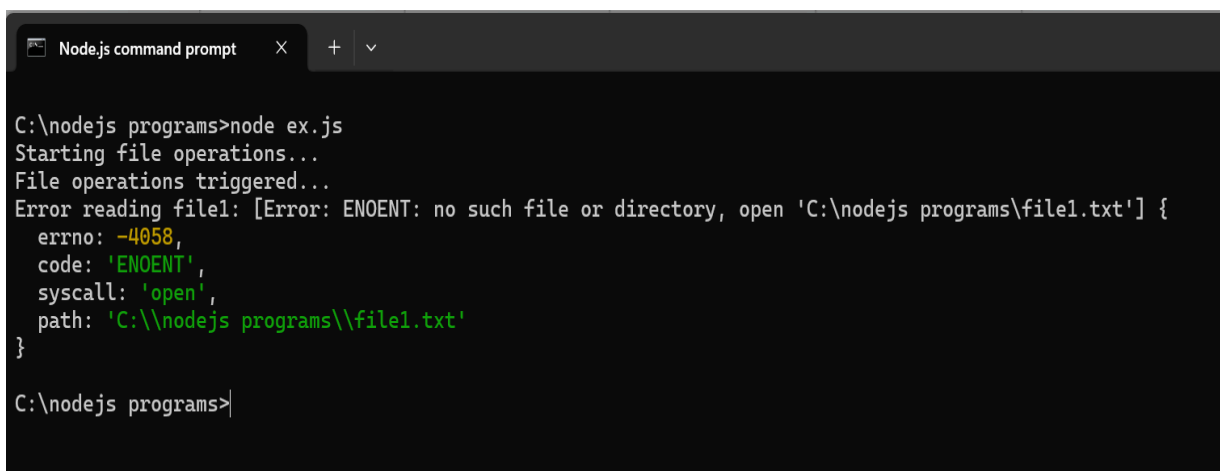
console.log("Starting file operations...");

fs.readFile('file1.txt', 'utf8', (err, data1) => {
  if (err) {
    console.error("Error reading file1:", err);
    return;
  }
  console.log("File1 content:", data1);

  fs.readFile('file2.txt', 'utf8', (err, data2) => {
    if (err) {
      console.error("Error reading file2:", err);
      return;
    }
    console.log("File2 content:", data2);

    fs.readFile('file3.txt', 'utf8', (err, data3) => {
      if (err) {
        console.error("Error reading file3:", err);
        return;
      }
      console.log("File3 content:", data3);
    });
  });
});
console.log("File operations triggered...");
```

Output:



```
Node.js command prompt x + v

C:\nodejs programs>node ex.js
Starting file operations...
File operations triggered...
Error reading file1: [Error: ENOENT: no such file or directory, open 'C:\nodejs programs\file1.txt'] {
  errno: -4058,
  code: 'ENOENT',
  syscall: 'open',
  path: 'C:\nodejs programs\file1.txt'
}

C:\nodejs programs>
```

- The file operations are executed asynchronously, but the structure of the code is deeply nested, making it look messy and harder to debug.

How to Avoid Callback Hell

Callback hell can be avoided using modern JavaScript techniques like Promises or Async/Await, which make the code cleaner, more readable, and easier to manage.

Solution 1: Using Promises

Promises provide a way to handle asynchronous operations in a chainable and readable manner. Here's how you can rewrite the above example using Promises:

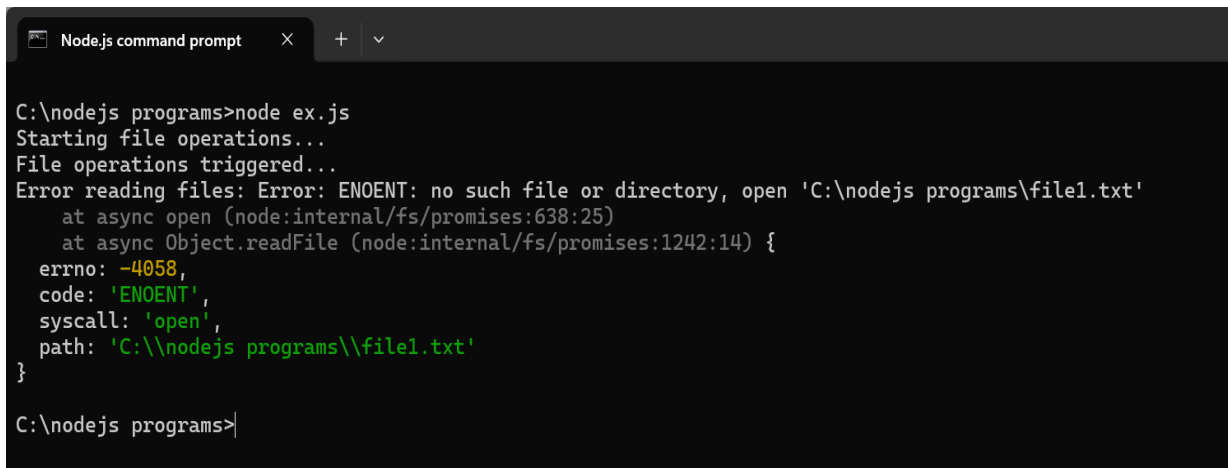
```
const fs = require('fs').promises;

console.log("Starting file operations...");

fs.readFile('file1.txt', 'utf8')
  .then(data1 => {
    console.log("File1 content:", data1);
    return fs.readFile('file2.txt', 'utf8');
  })
  .then(data2 => {
    console.log("File2 content:", data2);
    return fs.readFile('file3.txt', 'utf8');
  })
  .then(data3 => {
    console.log("File3 content:", data3);
  })
  .catch(err => {
    console.error("Error reading files:", err);
  });

console.log("File operations triggered...");
```

Output:



```
Node.js command prompt x + v

C:\nodejs programs>node ex.js
Starting file operations...
File operations triggered...
Error reading files: Error: ENOENT: no such file or directory, open 'C:\nodejs programs\file1.txt'
  at async open (node:internal/fs/promises:638:25)
  at async Object.readFile (node:internal/fs/promises:1242:14) {
  errno: -4058,
  code: 'ENOENT',
  syscall: 'open',
  path: 'C:\\nodejs programs\\file1.txt'
}

C:\nodejs programs>
```

Benefits:

- No nested structure.
- Error handling is centralized using `.catch()`.

Solution 2: Using Async/Await

Async/Await is built on top of Promises and makes the asynchronous code look synchronous, further improving readability.

```
const fs = require('fs').promises;

async function readFiles() {
  try {
    console.log("Starting file operations...");
```



```

const data1 = await fs.readFile('file1.txt', 'utf8');
console.log("File1 content:", data1);

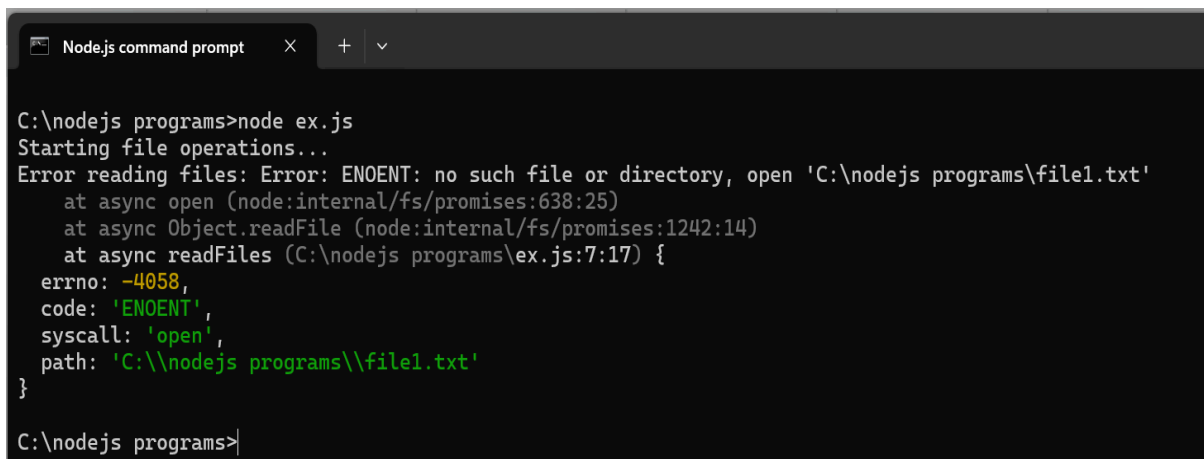
const data2 = await fs.readFile('file2.txt', 'utf8');
console.log("File2 content:", data2);

const data3 = await fs.readFile('file3.txt', 'utf8');
console.log("File3 content:", data3);

console.log("File operations completed...");
} catch (err) {
  console.error("Error reading files:", err);
}
}
readFiles();

```

Output:



```

Node.js command prompt
C:\nodejs programs>node ex.js
Starting file operations...
Error reading files: Error: ENOENT: no such file or directory, open 'C:\nodejs programs\file1.txt'
  at async open (node:internal/fs/promises:638:25)
  at async Object.readFile (node:internal/fs/promises:1242:14)
  at async readFiles (C:\nodejs programs\ex.js:7:17) {
  errno: -4058,
  code: 'ENOENT',
  syscall: 'open',
  path: 'C:\\nodejs programs\\file1.txt'
}
C:\nodejs programs>

```

Benefits:

- Read more like synchronous code.
- Error handling is simpler with try...catch.
- No nesting, making it clean and maintainable.

When to Use Each Approach

- Promises are good for chaining multiple asynchronous operations when simplicity is enough.
- Async/Await is more readable and preferred for complex workflows where maintaining sequence and error handling is critical.

Note:- We will discuss both of the above solutions in more detail in the next session.

Summary

- Callback Hell happens due to deeply nested callbacks in asynchronous programming.
- Reasons include poor readability, tightly coupled code, and difficult error handling.
- Avoid Callback Hell using:
 - Promises: For cleaner, chainable operations.
 - Async/Await: For synchronous-like flow and better readability.

By using modern techniques, you can write cleaner, more maintainable Node.js code that avoids callback hell and improves developer productivity.

Exercise:

Use GPT and explore real-world use cases of asynchronous programming in Node.js.

Put the below problem statement in the message box and see what ChatGPT says.

“Explain how asynchronous programming is used in real-world applications like file uploads, web scraping, or real-time chat systems. Provide examples with Node.js code.”

ChatGPT

Memory Full

Share

Explain how asynchronous programming is used in real-world applications like file uploads, web scraping, or real-time chat systems. Provide examples with Node.js code.

How Asynchronous Programming is Used in Real-World Applications

Asynchronous programming is a core strength of Node.js that enables handling multiple tasks efficiently without blocking the main thread. This feature is essential for building scalable and high-performance applications. Let's explore three common real-world use cases of asynchronous programming in Node.js:

1. File Uploads

When users upload files to a server, the server handles these uploads without blocking other requests. Asynchronous programming ensures that multiple file uploads can be processed simultaneously.

Example: Asynchronous File Upload

Message ChatGPT

+

Search

Reason

ChatGPT can make mistakes. Check important info.

After the 1.5 hr ILT, the student has to do a 30 minute live lab. Please refer to the Session 4 Lab doc in the LMS.